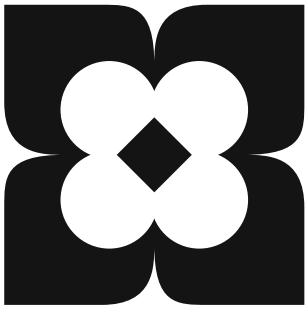
SECURITY OPERATIONS

Security Monitoring Foundation

Deployment & Operational Handover Report



TETRION

Four Minds. Infinite Possibilities.

Wazuh 4.14.7 / Windows & Linux endpoint coverage

PREPARED FOR TETRION

Prepared by Roman Rzayev

15 OCTOBER 2026 | TET-SOC-001 | VERSION 1.0

01 / MANAGEMENT OVERVIEW

Executive summary

Tetrion's security monitoring foundation brings a Windows workstation and an Ubuntu web server into one Wazuh monitoring platform. The supplied implementation record shows a running Wazuh 4.14.7 single-node stack, successful dashboard access and two active endpoint agents.

Windows Sysmon is installed with a community configuration, and Apache serves its default page on the Linux endpoint. Short bidirectional connectivity checks succeed between all three systems. These results establish the initial platform and endpoint-enrollment baseline.

Capability	Recorded result
Central monitoring	Three Wazuh containers Up; dashboard loads.
Endpoint coverage	win-client and ubuntu-web Active on 4.14.7.
Windows telemetry readiness	Sysmon64 Running; event ingestion still requires proof.
Web-service readiness	Apache Running; default page opens.
Operational acceptance	Controlled event validation, recovery and hardening evidence outstanding.

Management decision

Proceed with completion and acceptance testing. Do not treat the platform as production accepted until telemetry delivery, recovery, credentials and isolation controls are verified.

This is an evidence-based deployment report prepared for Tetrion. It is not a live audit of the original machines; the review is limited to the supplied screenshots and implementation notes.

02 / REPORT NAVIGATION

Contents & document control

Section	Page
Executive summary	2
Scope and assessment basis	4
Architecture & asset inventory	5
Deployment & telemetry design	6
Validation & acceptance priorities	7
Startup, shutdown & recovery	8
Evidence register & next actions	9
Evidence annex	10

Field	Value
Document ID	TET-SOC-001
Version / issue date	1.0 / 15 October 2026
Prepared for / prepared by	Tetrion / Roman Rzayev
Subject	Security Monitoring Foundation - Deployment Report
Status	Implementation recorded; acceptance actions open
Handling	Internal technical record; repository copy contains selected implementation evidence.

Source roles: Week1_SOC_Lab (1).docx defines the acceptance baseline; faridexiyetimbrat.docx supplies implementation notes and 29 screenshots. ED-Lab04-Executive-Report.pdf is a visual layout reference only. Its company identity, findings, targets and assessment claims are not carried into this report.

03 / SCOPE & INTERPRETATION

Scope and assessment basis

The scope is a three-system VMware environment: one Ubuntu monitoring server, one Ubuntu web endpoint and one Windows endpoint. The implementation uses a host-only network for internal communication and a second NAT interface on the monitoring server.

Included	Outside the demonstrated scope
VM resources, addressing and ICMP reachability	Comprehensive routing, firewall and time-synchronization audit
Wazuh stack startup and dashboard access	Continuous uptime, load testing and production high availability
Agent enrollment and point-in-time Active state	End-to-end validation of all required security events
Sysmon and Apache installation evidence	Incident response automation and attack detection effectiveness

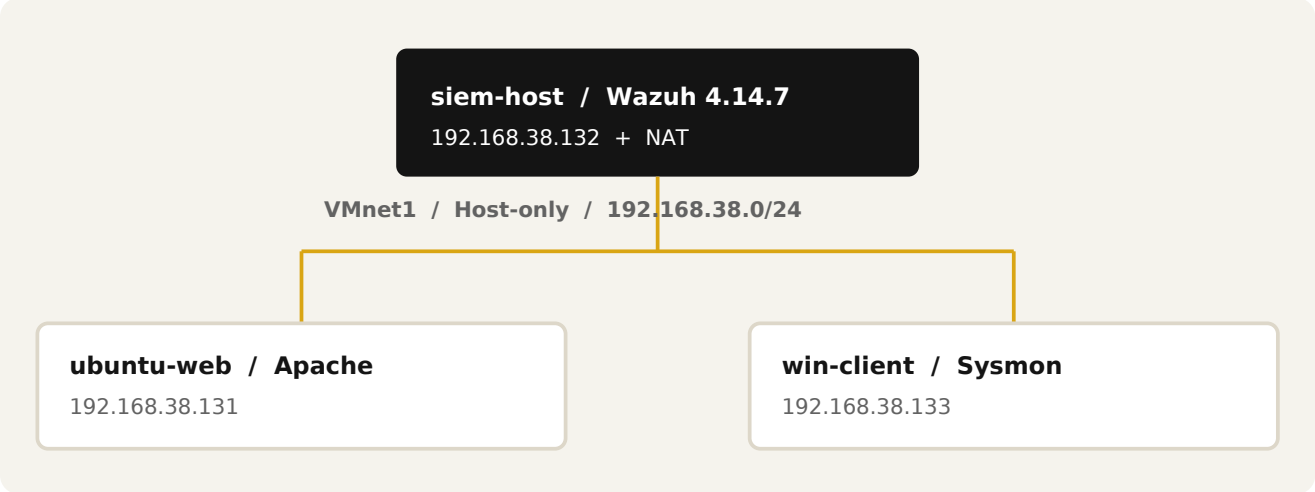
Evidence labels are intentionally precise. Recorded means a capture directly supports the stated observation. Partial means the capture supports only part of a requirement. Open means the supplied material does not establish completion. These labels are not results of new tests against the original systems.

> **Implementation variance**
>
> The endpoint resource allocations differ from the supplied baseline, and the Linux network captures show dynamically assigned addresses. These differences are documented rather than normalized into an assumed compliant configuration.

The report date is the document issue date, not an assertion about the date of every source capture. No passwords, private keys, certificates or invented operational measurements are included.

04 / SYSTEM DESIGN

Architecture & asset inventory



Asset / IP	Recorded resources	Role
siem-host 192.168.38.132	16 GB / 6 vCPU / 500 GB	Wazuh server; NAT also present
ubuntu-web 192.168.38.131	4 GB / 2 vCPU / 500 GB	Ubuntu 22.04; Apache; agent
win-client 192.168.38.133	4 GB / 4 vCPU / 60 GB	Windows 10 Enterprise Evaluation; agent; Sysmon

The screenshots associate the server with ens36 on 192.168.38.132/24 and ens33 on 192.168.121.156/24. The latter is the recorded NAT-side address and may change. VMnet1 DHCP is enabled; Linux interfaces carry the dynamic flag. Static assignment or reservation is therefore not established.

Recorded RAM allocation totals 24 GB, compared with the 22 GB baseline. Ubuntu has 4 GB instead of 2 GB; Windows has 4 vCPUs instead of 2 and a 60 GB disk instead of 80 GB. Both Linux virtual disks show 500 GB. The images do not establish physical disk consumption or available host capacity.

05 / PLATFORM IMPLEMENTATION

Deployment & telemetry design

The implementation clones the official wazuh-docker repository at v4.14.7, generates deployment certificates, and starts the single-node stack using Docker Compose. The visible services are wazuh.manager, wazuh.indexer and wazuh.dashboard; all three use 4.14.7 images and show Up status in the container listing.

Filebeat runs as part of the manager image in this deployment. A typical event path is: endpoint source -> Wazuh agent -> manager decoding and rule evaluation -> Filebeat alert forwarding -> indexer storage -> dashboard search. This is the expected design; the supplied captures do not trace one controlled event through every stage.

Component	Operational responsibility
Agent	Collect configured endpoint logs and forward them to the manager.
Manager	Decode events, evaluate rules and produce alerts.
Filebeat / indexer	Forward alert records and index them for search.
Dashboard	Authenticate analysts and present indexed monitoring information.

Windows enrollment first shows a service-start error and then a successful WazuhSvc start. A later inventory verifies win-client Active. Linux systemctl output verifies wazuh-agent Running; the final inventory verifies ubuntu-web Active. Both agent versions match the 4.14.7 manager line.

> **Visibility boundary**
>
> Sysmon64 Running proves installation, not ingestion. A Wazuh event containing the Microsoft-Windows-Sysmon/Operational channel, endpoint name and Event ID is still needed to demonstrate the Windows telemetry pipeline.

06 / OPERATIONAL ASSURANCE

Validation & acceptance priorities

Control	What the evidence establishes	Next verification
Connectivity	All six directed peer checks receive replies.	Record current clocks, routes and final adapter state.
Web service	Apache active; default page displayed.	Record the browser host and a dated HTTP response.
Agent coverage	Two Active endpoints on v4.14.7.	Recheck after stack restart and server reboot.
Security events	Dashboard overview only; no controlled-event records.	Generate and capture the five event cases below.
Credential control	Login is shown; password rotation is not.	Use the Docker deployment password-change procedure.

Required event cases are a failed SSH login, a sudo command, Linux local-user creation, a Windows process launch observed through Sysmon, and a failed Windows logon. Each needs the agent name, timestamp, rule description, rule level and screenshot. No rule IDs or levels have been invented in this report.

Priority before acceptance

Confirm persistent addressing and endpoint isolation; rotate default credentials; demonstrate five event cases and Sysmon ingestion; complete restart/reboot persistence checks; record memory and snapshots.

A short ping test proves sampled IP reachability. An Up container status proves a process was running at capture time. Neither alone establishes end-to-end security monitoring, sustained service health or recoverability.

07 / OPERATIONAL HANDOVER

Startup, shutdown & recovery

The following is the operating procedure for the recorded architecture. It is derived from the visible deployment path and normal Compose behavior; it has not been executed against the original VMs during this document review.

01 Start siem-host first. Confirm the internal interface is on 192.168.38.132/24 and that vm.max_map_count meets the Wazuh indexer prerequisite (at least 262144).

02 In the existing single-node deployment directory (recorded as /root/wazuh-docker/single-node), run docker compose up -d, then docker compose ps and inspect service logs if startup fails. Keep deployment credentials local.

03 Open the configured dashboard URL over HTTPS from the operator workstation. Verify authentication and index availability. Assess the deployment certificate warning using its actual issuer and subject; do not disable TLS verification as a routine workaround.

04 Start ubuntu-web and win-client. Verify Apache, Wazuh agent services and Sysmon64. Confirm both agents become Active and generate one authorized test event that appears in the dashboard.

05 For shutdown, stop endpoint workloads and shut down the endpoints first. On siem-host use docker compose stop before shutting down the VM. Do not use docker compose down -v or delete persistent volumes.

06 For recovery acceptance, record an agent inventory and a uniquely identifiable test alert, restart the stack, then reboot siem-host. Confirm the same agents and pre-existing alert remain searchable after each operation.

Record docker stats --no-stream at idle, the exact image versions, snapshot names and Windows evaluation expiry. Stop/start, reboot persistence and memory measurements remain unverified until their outputs are retained.

08 / EVIDENCE-LED HANDOVER

Evidence register & next actions

The following annex retains all 29 embedded screenshots in source order, with one observation per figure. Every screenshot sits in a consistent 493 x 205 pt frame, is fitted proportionally without cropping, and retains its original embedded pixel data. Smaller captures cannot recover detail absent from the original.

Evidence range	Coverage
E01-E07	VMware resources, host-only network and interface addresses
E08-E10	Peer reachability in all three directions
E11-E14	Sysmon installation and Apache service
E15-E20	Wazuh release, certificates, containers and dashboard
E21-E26	Windows and Linux enrollment and Active inventory
E27-E29	System naming commands and Windows identity context

For review at source resolution, use evidence/screenshots/ in the accompanying repository. evidence/manifest.json records dimensions and SHA-256 digests. The editable document is a companion; the PDF is the fixed-layout presentation master.

> **Close-out deliverables**
>
> Complete the acceptance matrix in docs/acceptance.md using new screenshots and outputs from the actual VMs. The retrospective journal distinguishes observations from unrecorded troubleshooting; missing events and measurements must be supplied by execution, not reconstruction.

Official technical references: documentation.wazuh.com/current/deployment-options/docker/wazuh-container.html; docs.docker.com/engine/install/ubuntu/; learn.microsoft.com/sysinternals/downloads/sysmon; github.com/SwiftOnSecurity/sysmon-config. These are implementation references, not claims that all documented steps were performed.

EVIDENCE ANNEX

Compute provisioning

E01 Windows endpoint resources

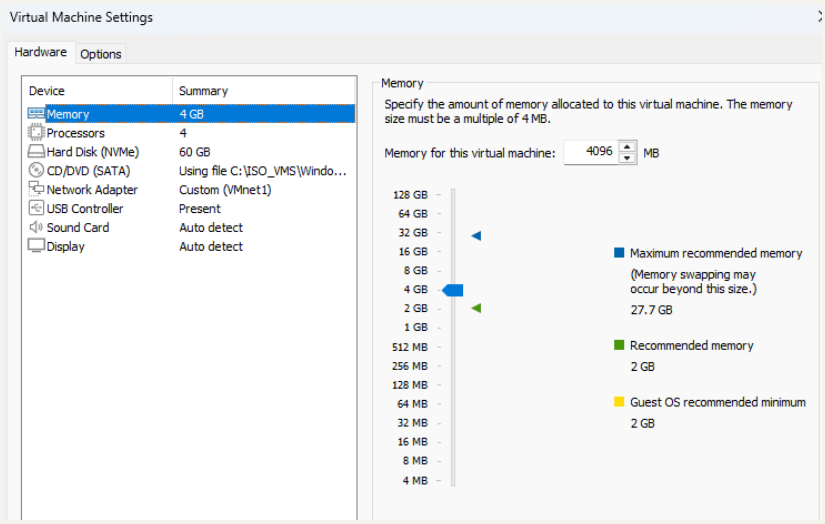


Figure E01. VMware settings show 4 GB RAM, 4 vCPUs, a 60 GB disk and Custom (VMnet1) networking.

Source: image1.png | 738 x 464 px | Original pixels retained

E02 Linux endpoint resources

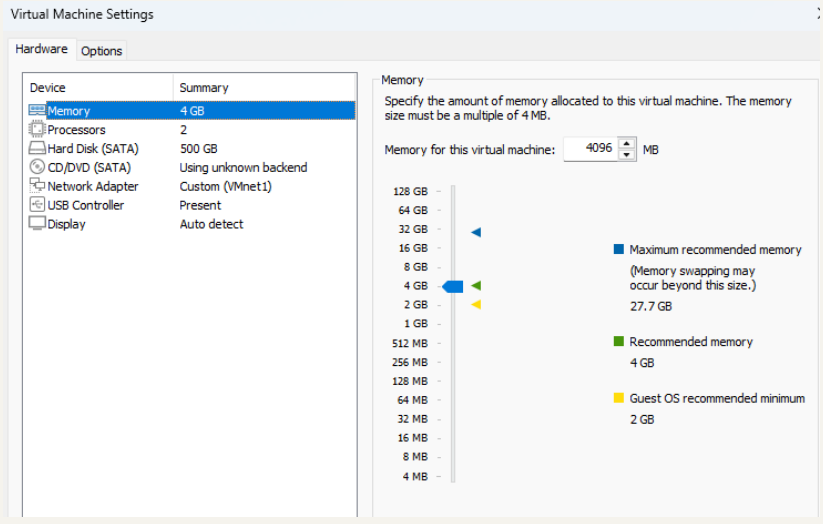


Figure E02. VMware settings show 4 GB RAM, 2 vCPUs, a 500 GB disk and Custom (VMnet1) networking.

Source: image2.png | 737 x 465 px | Original pixels retained

EVIDENCE ANNEX

Server & virtual networking

E03 Monitoring server resources

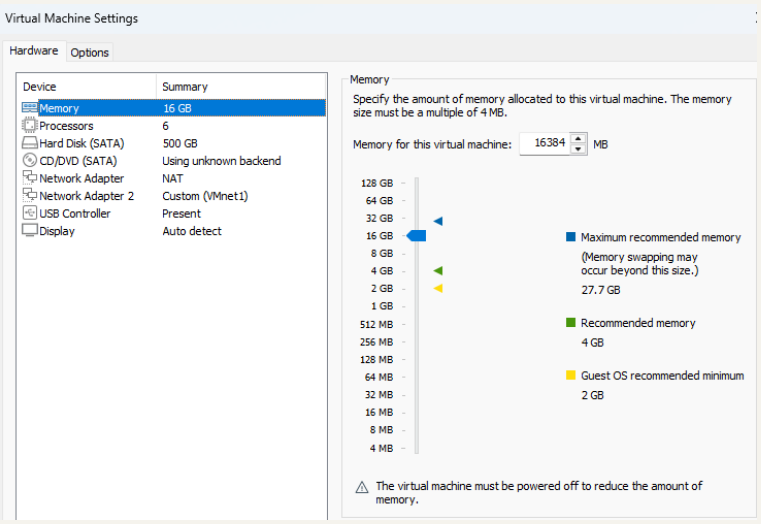


Figure E03. VMware settings show 16 GB RAM, 6 vCPUs, a 500 GB disk, NAT and Custom (VMnet1) adapters.

Source: image3.png | 735 x 500 px | Original pixels retained

E04 Virtual network configuration

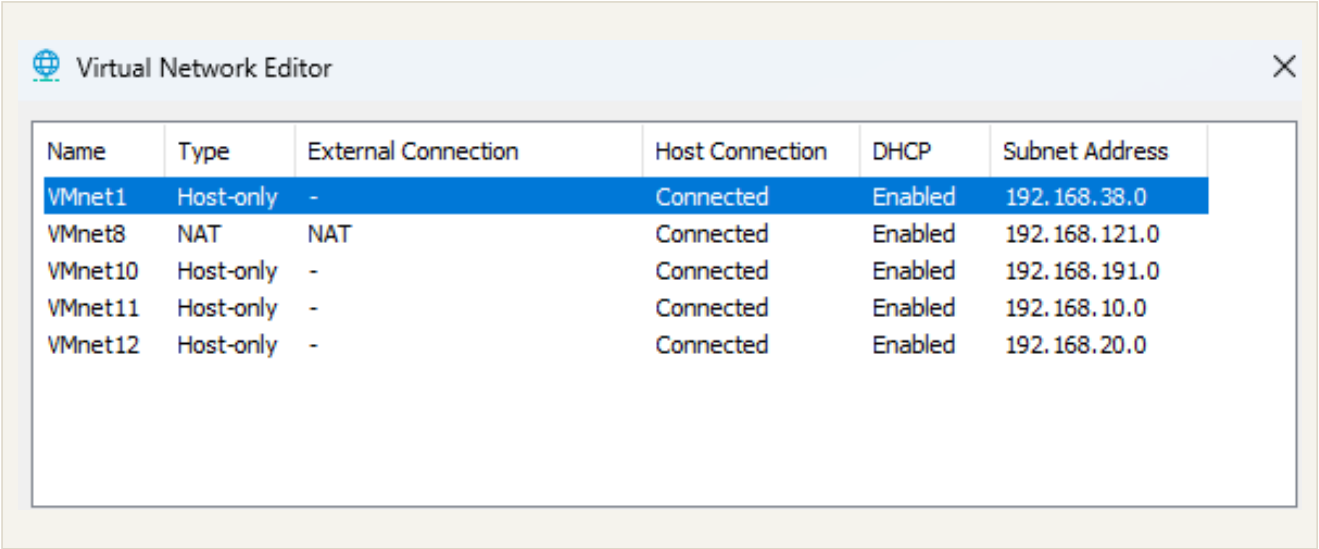


Figure E04. VMnet1 is Host-only on 192.168.38.0; DHCP is enabled. VMnet8 provides a separate NAT network.

Source: image4.png | 591 x 216 px | Original pixels retained

EVIDENCE ANNEX

Endpoint addressing

E05 Windows address configuration

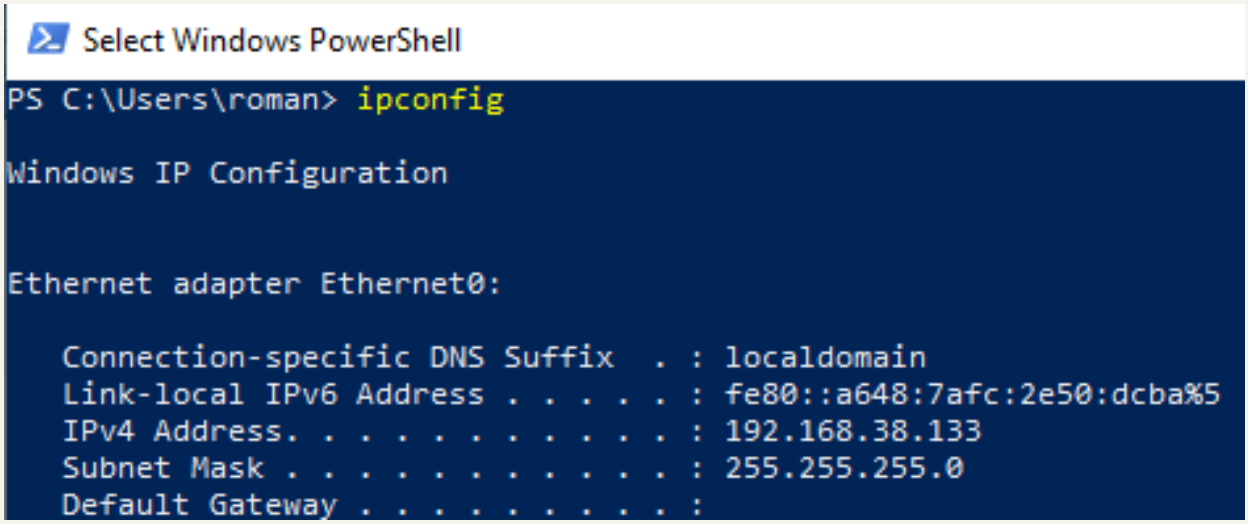


Figure E05. The Windows endpoint reports 192.168.38.133 with subnet mask 255.255.255.0 and no displayed default gateway.

Source: image5.png | 472 x 196 px | Original pixels retained

E06 Linux endpoint address configuration

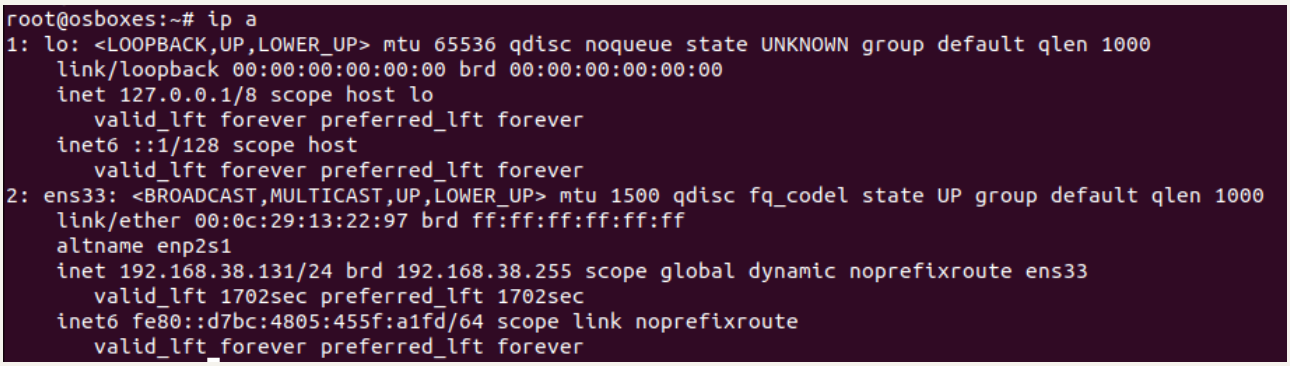


Figure E06. The Linux endpoint reports 192.168.38.131/24. The dynamic flag means static addressing is not demonstrated.

Source: image6.png | 918 x 255 px | Original pixels retained

EVIDENCE ANNEX

Server network & reachability

E07 Monitoring server interfaces

```
root@osboxes:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:60:9c:c1 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.121.156/24 brd 192.168.121.255 scope global dynamic noprefixroute ens33
       valid_lft 1669sec preferred_lft 1669sec
    inet6 fe80::12a0:cc38:f0c3:89a/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:60:9c:cb brd ff:ff:ff:ff:ff:ff
    altname enp2s4
    inet 192.168.38.132/24 brd 192.168.38.255 scope global dynamic noprefixroute ens36
       valid_lft 1669sec preferred_lft 1669sec
    inet6 fe80::95fa:dd2c:72e8:ac67/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Figure E07. The server reports 192.168.38.132/24 on ens36 and NAT address 192.168.121.156/24 on ens33; both are marked dynamic.

Source: image7.png | 913 x 380 px | Original pixels retained

E08 Server-to-endpoint reachability

```
root@osboxes:~# ping 192.168.38.131
PING 192.168.38.131 (192.168.38.131) 56(84) bytes of data.
64 bytes from 192.168.38.131: icmp_seq=1 ttl=64 time=0.764 ms
64 bytes from 192.168.38.131: icmp_seq=2 ttl=64 time=0.958 ms
64 bytes from 192.168.38.131: icmp_seq=3 ttl=64 time=0.833 ms
^C
--- 192.168.38.131 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2034ms
rtt min/avg/max/mdev = 0.764/0.851/0.958/0.080 ms
root@osboxes:~# ping 192.168.38.133
PING 192.168.38.133 (192.168.38.133) 56(84) bytes of data.
64 bytes from 192.168.38.133: icmp_seq=1 ttl=128 time=0.760 ms
64 bytes from 192.168.38.133: icmp_seq=2 ttl=128 time=0.942 ms
64 bytes from 192.168.38.133: icmp_seq=3 ttl=128 time=0.807 ms
^C
--- 192.168.38.133 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
```

Figure E08. The monitoring server receives three replies from each endpoint, with 0% packet loss in both short tests.

Source: image8.png | 570 x 312 px | Original pixels retained

EVIDENCE ANNEX

Peer connectivity

E09 Linux-to-peer reachability

```
root@osboxes:~# ping 192.168.38.132
PING 192.168.38.132 (192.168.38.132) 56(84) bytes of data.
64 bytes from 192.168.38.132: icmp_seq=1 ttl=64 time=0.366 ms
64 bytes from 192.168.38.132: icmp_seq=2 ttl=64 time=1.10 ms
64 bytes from 192.168.38.132: icmp_seq=3 ttl=64 time=0.774 ms
^C
--- 192.168.38.132 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2016ms
rtt min/avg/max/mdev = 0.366/0.745/1.095/0.298 ms
root@osboxes:~# ping 192.168.38.133
PING 192.168.38.133 (192.168.38.133) 56(84) bytes of data.
64 bytes from 192.168.38.133: icmp_seq=1 ttl=128 time=0.966 ms
64 bytes from 192.168.38.133: icmp_seq=2 ttl=128 time=0.890 ms
64 bytes from 192.168.38.133: icmp_seq=3 ttl=128 time=0.751 ms
^C
--- 192.168.38.133 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.751/0.869/0.966/0.089 ms
```

Figure E09. The Linux endpoint receives three replies from the server and three from Windows, with 0% packet loss.

Source: image9.png | 577 x 331 px | Original pixels retained

E10 Windows-to-peer reachability

```
PS C:\Users\roman> ping 192.168.38.131

Pinging 192.168.38.131 with 32 bytes of data:
Reply from 192.168.38.131: bytes=32 time<1ms TTL=64
Reply from 192.168.38.131: bytes=32 time<1ms TTL=64
Reply from 192.168.38.131: bytes=32 time<1ms TTL=64
Reply from 192.168.38.131: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.38.131:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
PS C:\Users\roman> ping 192.168.38.132

Pinging 192.168.38.132 with 32 bytes of data:
Reply from 192.168.38.132: bytes=32 time<1ms TTL=64
Reply from 192.168.38.132: bytes=32 time<1ms TTL=64
Reply from 192.168.38.132: bytes=32 time=1ms TTL=64
Reply from 192.168.38.132: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.38.132:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms
```

Figure E10. Windows receives four replies from the Linux endpoint and four from the server, with 0% packet loss.

Source: image10.png | 403 x 340 px | Original pixels retained

EVIDENCE ANNEX

Endpoint service preparation

E11 Windows Sysmon installation

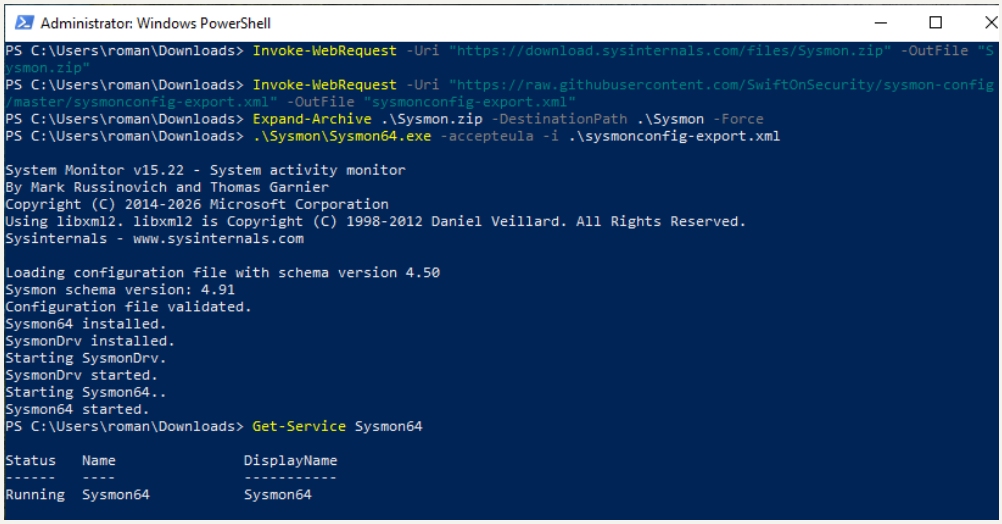


Figure E11. Sysmon v15.22 loads the SwiftOnSecurity configuration; the Sysmon64 service is Running. Dashboard ingestion is not shown.

Source: image11.png | 817 x 423 px | Original pixels retained

E12 Apache package installation

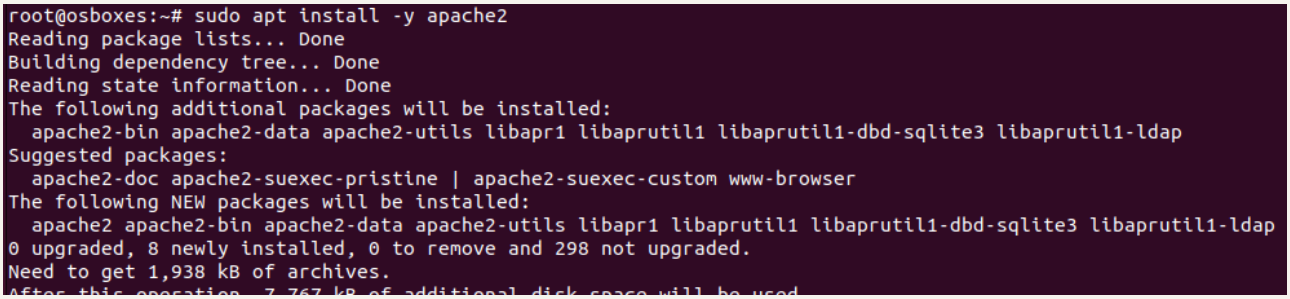


Figure E12. APT output shows installation of Apache packages on the Linux endpoint; this capture is an installation stage.

Source: image12.png | 994 x 225 px | Original pixels retained

EVIDENCE ANNEX

Web-service readiness

E13 Apache service state

```
root@osboxes:~# sudo systemctl enable --now apache2
Synchronizing state of apache2.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable apache2
root@osboxes:~# systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
     Active: active (running) since Fri 2026-10-09 11:01:09 EDT; 45s ago
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 4151 (apache2)
      Tasks: 55 (limit: 4541)
     Memory: 5.3M
        CPU: 18ms
     CGroup: /system.slice/apache2.service
             ├─4151 /usr/sbin/apache2 -k start
             ├─4152 /usr/sbin/apache2 -k start
             └─4153 /usr/sbin/apache2 -k start

Oct 09 11:01:09 osboxes systemd[1]: Starting The Apache HTTP Server...
Oct 09 11:01:09 osboxes apachectl[4150]: AH00558: apache2: Could not reliably determine the server's fully
Oct 09 11:01:09 osboxes systemd[1]: Started The Apache HTTP Server.
```

Figure E13. Apache is enabled and active (running). A ServerName warning is visible; the service remains running.

Source: image13.png | 967 x 358 px | Original pixels retained

E14 Web-service functional check

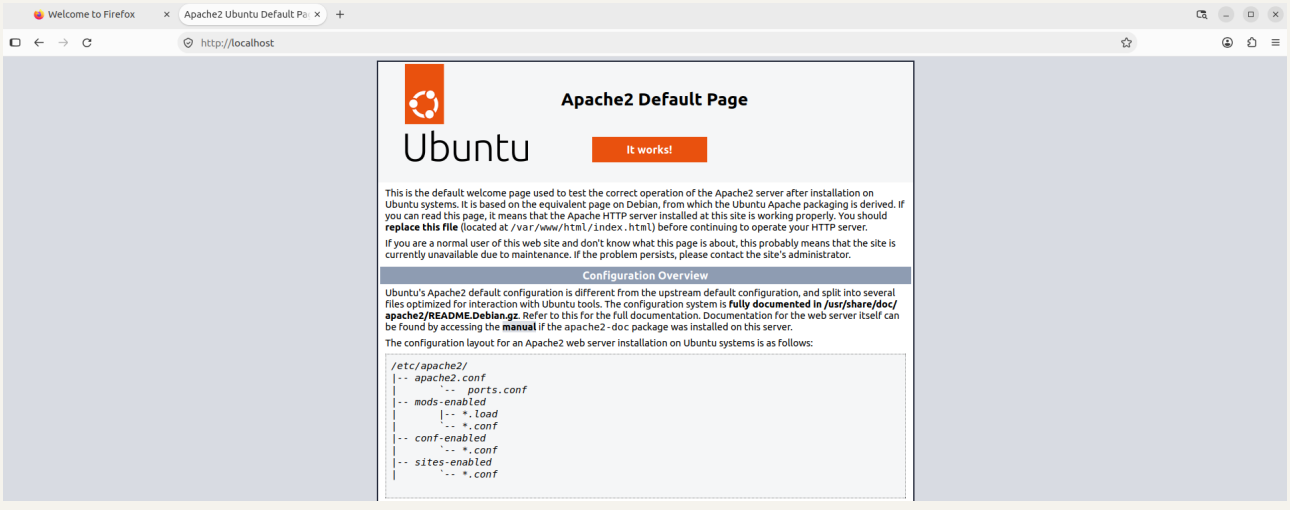


Figure E14. A browser displays the Apache2 Ubuntu Default Page at the Linux endpoint address. Browser host identity is not independently shown.

Source: image14.png | 1920 x 744 px | Original pixels retained

EVIDENCE ANNEX

Wazuh release & certificates

E15 Pinned Wazuh source

```
root@osboxes:~# git clone https://github.com/wazuh/wazuh-docker.git -b v4.14.7 --depth=1
cd wazuh-docker/single-node
pwd
Cloning into 'wazuh-docker'...
remote: Enumerating objects: 152, done.
remote: Counting objects: 100% (152/152), done.
remote: Compressing objects: 100% (121/121), done.
remote: Total 152 (delta 23), reused 83 (delta 19), pack-reused 0 (from 0)
Receiving objects: 100% (152/152), 93.69 KiB | 116.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.
Note: switching to 'adcc5b57d2f7edfcbe6c399272dc76fbdf12b623'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:
```

Figure E15. The recorded git clone selects wazuh-docker tag v4.14.7. Detached HEAD is expected for a release tag.

Source: image15.png | 817 x 364 px | Original pixels retained

E16 Certificate-generation preparation

```
Turn off this advice by setting config variable advice.detachedHead to false

/root/wazuh-docker/single-node
root@osboxes:~/wazuh-docker/single-node# cd ~/wazuh-docker/single-node
pwd
/root/wazuh-docker/single-node
root@osboxes:~/wazuh-docker/single-node# docker compose -f generate-indexer-certs.yml run --rm generator
ls config/wazuh_indexer_ssl_certs/
[+] run 1/1
 ✔Network single-node_default Created
Image wazuh/wazuh-certs-generator:0.0.4 Pulling
cf48939c4ae7 Pulling fs layer 0B
f0d8a57b0a96 Pulling fs layer 0B
035f2e5bf75c Pulling fs layer 0B
df36708a1273 Pulling fs layer 0B
df36708a1273 Download complete 0B
cf48939c4ae7 Download complete 0B
f0d8a57b0a96 Downloading 1.049MB
f0d8a57b0a96 Downloading 2.097MB
f0d8a57b0a96 Downloading 3.146MB
f0d8a57b0a96 Downloading 4.194MB
f0d8a57b0a96 Downloading 6.291MB
```

Figure E16. The certificate-generation Compose command pulls the Wazuh certificate-tool image for the single-node deployment.

Source: image16.png | 950 x 432 px | Original pixels retained

EVIDENCE ANNEX

Certificate completion & runtime

E17 Certificate-generation result

```
Image wazuh/wazuh-certs-generator:0.0.4 Pulled
Container single-node-generator-run-7664e24fc99b Creating
Container single-node-generator-run-7664e24fc99b Created
Checking https://packages.wazuh.com/4.14/wazuh-certs-tool.sh ...
Downloaded wazuh-certs-tool.sh from https://packages.wazuh.com/4.14/
09/10/2026 15:16:38 INFO: Verbose logging redirected to //wazuh-certificates-tool.log
09/10/2026 15:16:38 INFO: Generating the root certificate.
09/10/2026 15:16:38 INFO: Generating Admin certificates.
09/10/2026 15:16:38 INFO: Admin certificates created.
09/10/2026 15:16:38 INFO: Generating Wazuh indexer certificates.
09/10/2026 15:16:38 INFO: Wazuh indexer certificates created.
09/10/2026 15:16:38 INFO: Generating Filebeat certificates.
09/10/2026 15:16:38 INFO: Wazuh Filebeat certificates created.
09/10/2026 15:16:38 INFO: Generating Wazuh dashboard certificates.
09/10/2026 15:16:39 INFO: Wazuh dashboard certificates created.
/wazuh-certs-tool.sh: line 636: find: command not found
Moving created certificates to the destination directory
Changing certificate permissions
Setting UID indexer and dashboard
Setting UID for wazuh manager and worker
admin-key.pem  root-ca-manager.key  wazuh.dashboard-key.pem  wazuh.indexer.pem
admin.pem      root-ca-manager.pem  wazuh.dashboard.pem      wazuh.manager-key.pem
root-ca.key    root-ca.pem          wazuh.indexer-key.pem    wazuh.manager.pem
root@osboxes:~/wazuh-docker/single-node#
```

Figure E17. The tool reports certificates created for indexer, manager, Filebeat and dashboard, and moves them into the deployment directory.

Source: image17.png | 781 x 464 px | Original pixels retained

E18 Running single-node stack

```
root@osboxes:~/wazuh-docker/single-node# docker compose up -d
docker compose ps
[+] up 63/63
 ✔Image wazuh/wazuh-dashboard:4.14.7         Pulled                                    110.6s
 ✔Image wazuh/wazuh-manager:4.14.7           Pulled                                    132.8s
 ✔Image wazuh/wazuh-indexer:4.14.7           Pulled                                    122.9s
 ✔Volume single-node_filebeat_etc            Created                                     0.0s
 ✔Volume single-node_wazuh_etc               Created                                     0.0s
 ✔Volume single-node_wazuh_agentless         Created                                     0.0s
 ✔Volume single-node_wazuh-indexer-data      Created                                     0.0s
 ✔Volume single-node_filebeat_var            Created                                     0.0s
 ✔Volume single-node_wazuh_active_response   Created                                     0.0s
 ✔Volume single-node_wazuh-dashboard-config  Created                                     0.0s
 ✔Volume single-node_wazuh-dashboard-custom  Created                                     0.0s
 ✔Volume single-node_wazuh_var_multigroups   Created                                     0.0s
 ✔Volume single-node_wazuh_integrations      Created                                     0.0s
 ✔Volume single-node_wazuh_queue             Created                                     0.0s
 ✔Volume single-node_wazuh_logs              Created                                     0.0s
 ✔Volume single-node_wazuh_wodles            Created                                     0.0s
 ✔Volume single-node_wazuh_api_configuration Created                                     0.0s
 ✔Container single-node-wazuh.indexer-1      Started                                     8.9s
 ✔Container single-node-wazuh.manager-1      Started                                     9.3s
 ✔Container single-node-wazuh.dashboard-1    Started                                     1.1s
NAME                       IMAGE                          COMMAND                  SERVICE           CREATED          STATUS
single-node-wazuh.dashboard-1   wazuh/wazuh-dashboard:4.14.7   "/entrypoint.sh"         wazuh.dashboard   1 second ago     Up Less than a second
single-node-wazuh.indexer-1     wazuh/wazuh-indexer:4.14.7     "/entrypoint.sh open…"   wazuh.indexer     10 seconds ago   Up 1 second
single-node-wazuh.manager-1     wazuh/wazuh-manager:4.14.7     "/init"                  wazuh.manager     10 seconds ago   Up Less than a second
::]:55000->55000/tcp, 1516/tcp
root@osboxes:~/wazuh-docker/single-node#
```

Figure E18. Docker Compose lists the dashboard, indexer and manager with 4.14.7 images and Up status. This is a point-in-time startup check.

Source: image18.png | 1314 x 556 px | Original pixels retained

EVIDENCE ANNEX

Dashboard access

E19 Dashboard sign-in page

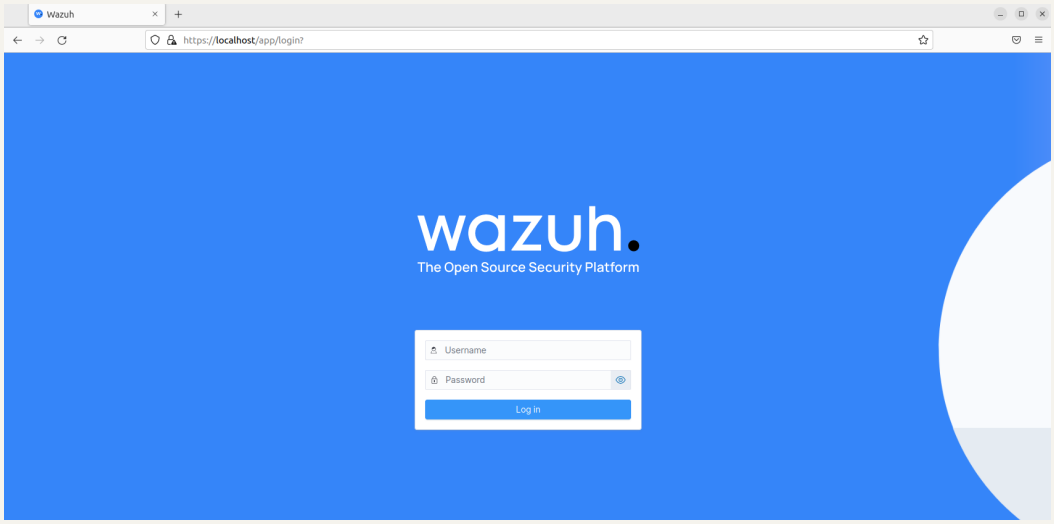


Figure E19. The Wazuh sign-in page is accessible in a browser. This does not prove a password change or certificate trust configuration.

Source: image19.png | 1698 x 836 px | Original pixels retained

E20 Authenticated dashboard

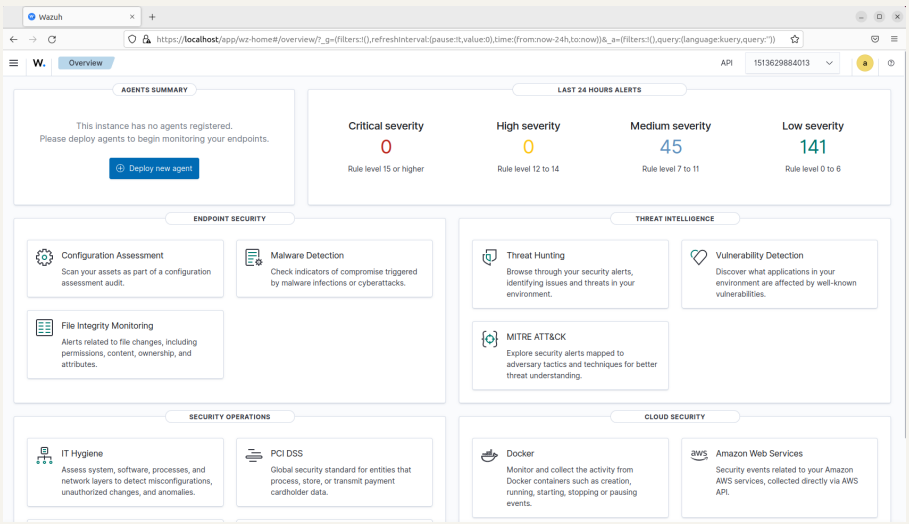


Figure E20. The Wazuh overview loads after authentication and shows security-event summary widgets. These are not the five required controlled tests.

Source: image20.png | 1698 x 969 px | Original pixels retained

EVIDENCE ANNEX

Windows agent enrollment

E21 Windows enrollment settings

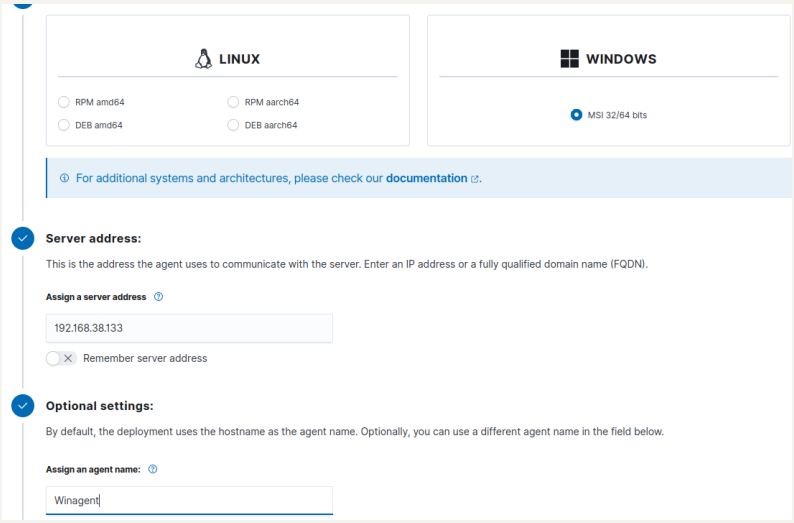


Figure E21. The dashboard deployment wizard selects Windows and the manager address 192.168.38.132 for an agent installation.

Source: image21.png | 1100 x 718 px | Original pixels retained

E22 Windows agent service start

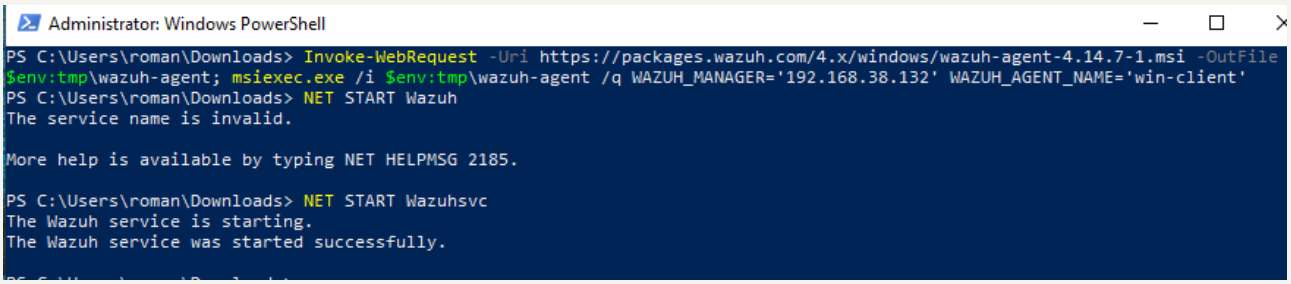


Figure E22. The agent install command names win-client. An initial NET HELPMSG 2185 response is followed by a successful WazuhSvc start.

Source: image22.png | 876 x 187 px | Original pixels retained

EVIDENCE ANNEX

Windows inventory & Linux enrollment

E23 Windows agent active

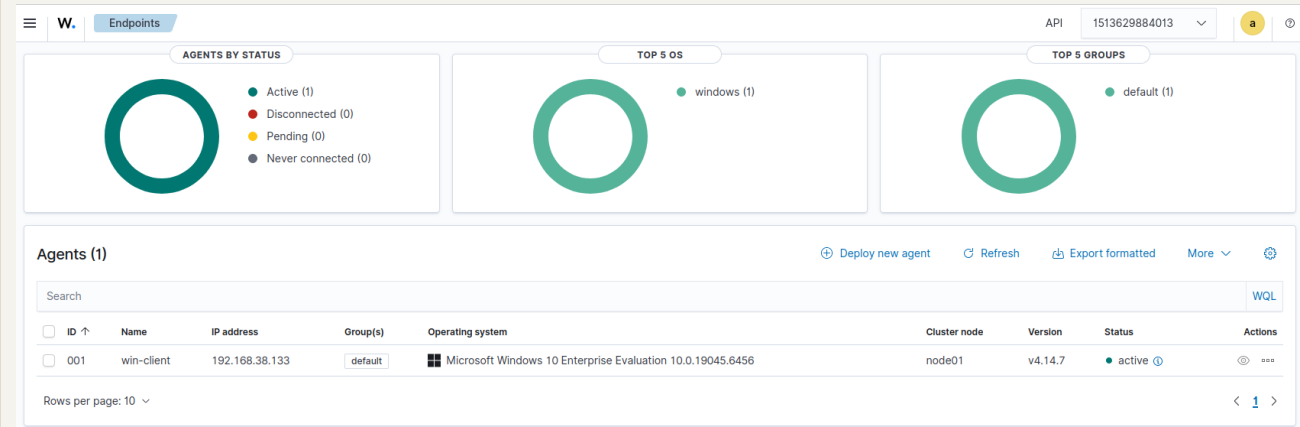


Figure E23. The inventory lists win-client at 192.168.38.133, agent v4.14.7, with active status.

Source: image23.png | 1681 x 553 px | Original pixels retained

E24 Linux enrollment settings

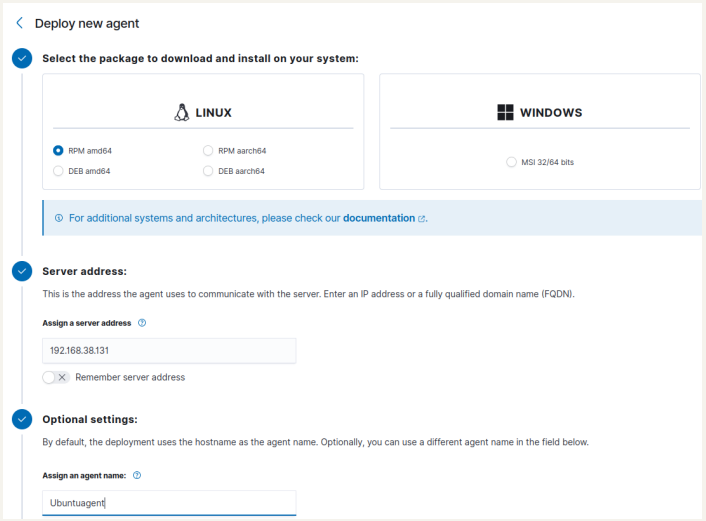


Figure E24. The dashboard deployment wizard selects Linux and the same monitoring-server address.

Source: image24.png | 1101 x 812 px | Original pixels retained

EVIDENCE ANNEX

Linux service & endpoint coverage

E25 Linux agent service state

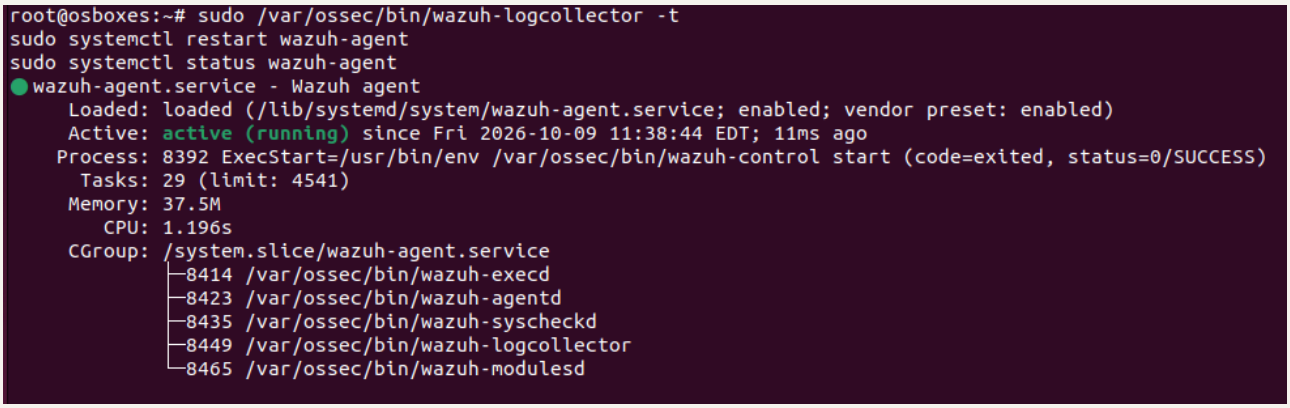


Figure E25. systemctl reports wazuh-agent enabled and active (running), with agent processes visible.

Source: image25.png | 983 x 305 px | Original pixels retained

E26 Two active monitored endpoints

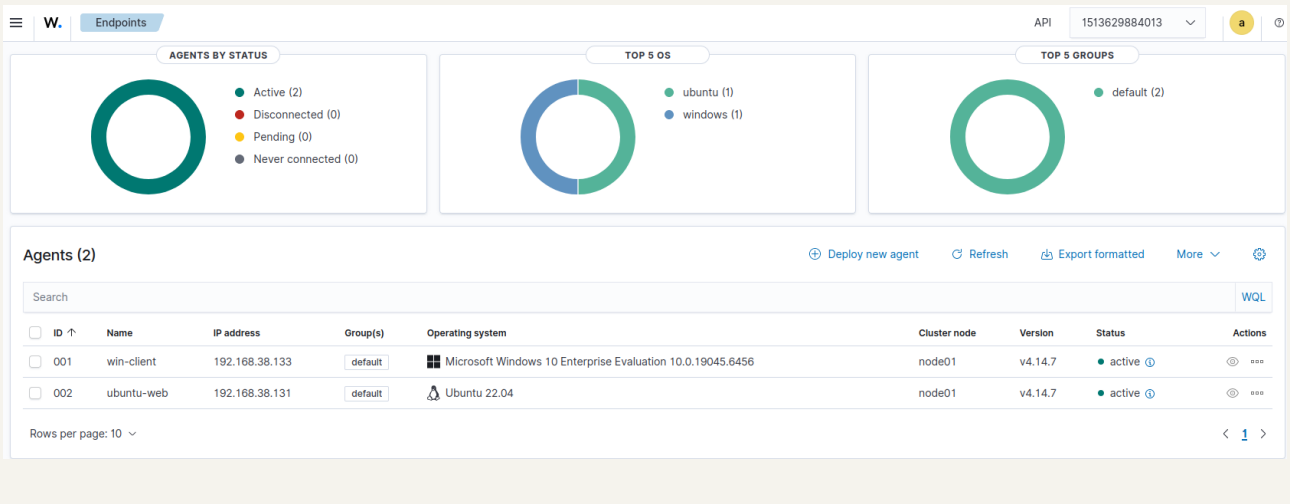


Figure E26. The inventory lists win-client and ubuntu-web as active, both on v4.14.7. The Linux endpoint is Ubuntu 22.04.

Source: image26.png | 1677 x 592 px | Original pixels retained

EVIDENCE ANNEX

Asset naming

E27 Monitoring-server naming

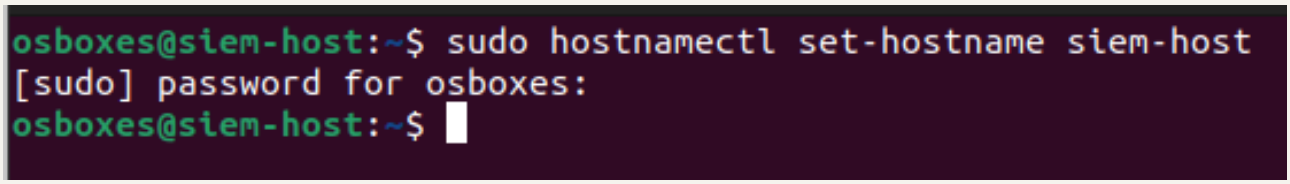


Figure E27. The hostnamectl command sets the server hostname to siem-host. A separate post-change hostname query is not shown.

Source: image27.png | 559 x 76 px | Original pixels retained

E28 Linux endpoint naming

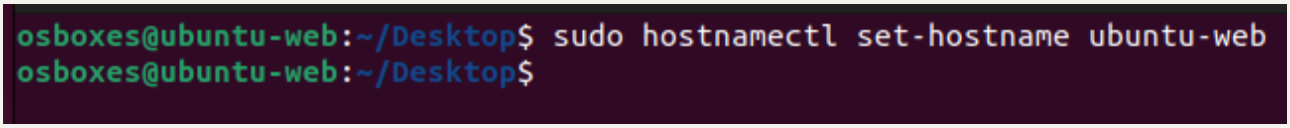


Figure E28. The hostnamectl command sets the Linux endpoint hostname to ubuntu-web.

Source: image28.png | 647 x 60 px | Original pixels retained

EVIDENCE ANNEX

Windows identity context

E29 Windows hostname context

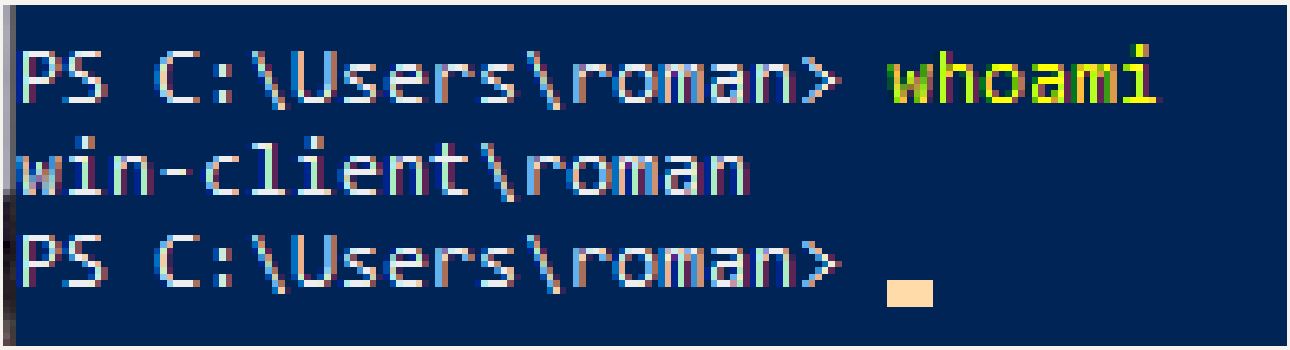


Figure E29. whoami returns win-client\roman, supporting the Windows computer-name context.

Source: image29.png | 196 x 52 px | Original pixels retained